Quantum low-energy and time-evolution approximations from a machine learning perspective
FYS5429

Amund Solum Alsvik
University of Oslo
(Dated: June 5, 2026)

We study parametric matrix models (PMMs) as low dimensional machine learning surrogates for many body Hamiltonian systems. The main tasks are static prediction of low-lying energies and time-evolution of quantum states in the Lipkin and pairing models. We also introduce a compression method for finding low dimensional subspaces that preserve low-energy structure over a parameter domain. The compression method is used both as a tool for reducing trained PMM surrogates and as a way of solving a dimensional mismatch problem in time-evolution experiments. For the Lipkin model, the PMM performs better compared to a neural network baseline in test accuracy, but requires relatively large dimensions and significantly longer training times. The pairing model gives stronger results. In the static spectral task, PMMs achieve lower test errors than the neural network baseline with comparable training time, while using substantially fewer trainable parameters. The strongest results are the time-evolution results in the pairing model. For both survival probability and pair-occupation observables, PMMs substantially outperform a GRU baseline with much fewer trainable parameters, using the compression method as a benchmark. In the fixed observable setting, the PMM learns time-evolution of a compressed observable, rather than using a fully learned readout. These results suggest that PMMs, combined with compression, can provide effective low dimensional representations of nontrivial many body time-evolution.

ACKNOWLEDGEMENT

Many thanks to my supervisor Morten for providing code for implementation of the Lipkin and pairing model, a framework code for implementing the PMMs and for useful discussions.

CONTENTS

1. INTRODUCTION

Many body quantum systems are central in quantum physics and quantum computing. In quantum control theory, where the aim is to steer quantum systems toward desired states or dynamical behavior by varying external parameters, one often requires repeated evaluations of parameter dependent Hamiltonian, time-evolved states and expectation values (Dong and Petersen [1]). As explained in Atanasova et al. [2], one of the biggest hurdles for performing these computations is the so called "dimensionality curse", meaning that the degrees of freedom in the Hilbert space grows rapidly with the size of the system. In light of this we explore the applicability of a promising new machine learning method, known as parametric matrix models (PMMs), introduced by Cook et al. [3]. Specifically, we investigate whether

PMMs are able to approximate many body quantities, such as the low-lying energy spectrum as well as the time-evolution of many body systems. To do this, we work with two common toy models, namely the Lipkin model and the pairing model, both having parameter dependent Hamiltonians. These models are widely used as testbeds, due to their relatively simple structure, along with availability of exact numerical solutions in the regimes considered here. Machine learning methods such as PMMs can alleviate the cost of repeated exact calculations over the parameter domain, by learning a significantly smaller surrogate Hamiltonians. In practice, however, selecting the dimension for the PMM is not always straightforward. If one uses too few dimensions, the model may not be expressive enough, while too many dimensions can lead to longer training times, as well as diminishing the computational gain of reducing dimensionality. In this report, we therefore introduce a compression method for extracting a low dimensional subspace to represent the low-energy behavior of the model. This method plays two roles in this report. First, it helps us to identify whether a trained PMM has directions that are less important for the low-lying spectrum, allowing further reduction of the surrogate dimension. Second, it is used as a tool for solving a dimensional mismatch problem, that arises when training the PMM for time-evolution tasks with fixed observables.

The structure of the report is as follows. In section 2 we briefly discuss the relevant many body models and their Hamiltonians. Then, in section 3, we introduce the affine eigenvalue PMM method and its architecture. Following in section 4, we introduce the compression algorithm and explain how it is used in this report. In section 5, we discuss the implementation choices, before moving to section 6, where we present our empirical results on the Lipkin and pairing models and compare with neural network baselines. Finally, in section 7, we finish with a conclusion and directions for further work.

2. PHYSICAL MODELS AND LEARNING TASKS

In this section we introduce the Lipkin and pairing model, along with the quantities we want our model to approximate. A full treatment of the physical models are beyond the scope of this report, so we restrict ourselves to the concepts needed to understand the exact machine learning tasks and what makes then challenging. The explanations of the models in this section are mostly adapted from Hjorth-Jensen [4] [5] and Cervia et al. [6]. These sources also contain a more detailed explanation of the underlying physics.

2.1. The Lipkin model

There are several variants and formulations the Lipkin model, along with physical interpretations. For the scope of this report, we focus on one example, without diving too deep into the physics. We state the premise mathematically, focusing only on the aspects that are important to the applications of PMMs. For a parameter domain $[a, b]$, the Lipkin Hamiltonian is given by

$$H(V) = \epsilon J_z + \frac{V}{N}(J_x^2 - J_y^2),$$

where J_z, J_x and J_y are Hermitian collective spin operators, $V \in [a, b]$ controls the interaction strength and ϵ sets the energy scale of the non interacting part of the Hamiltonian. In this report we set $\epsilon = 1$, which is a standard normalization and does not affect the qualitative behavior of the model. For N spin-1/2 particles, the full Hilbert space has dimension $\dim\big((\mathbb{C}^2)^{\otimes N}\big) = 2^N$. The Hamiltonian commutes with the total spin operator $J^2 = J_x^2 + J_y^2 + J_z^2$, which allows us to decompose the Hilbert space into invariant subspaces labeled by the total spin J. For spin-1/2 specifically, we label

$$J = \frac{N}{2}, \frac{N}{2} - 1, \ldots,$$

where the largest total spin is $J = N/2$. This subspace contains the states that are symmetric under permutation of tensor factors and is invariant under H. In this report we restrict ourselves to this subspace which has dimension $N + 1$, which is a standard treatment of the model. Hence, when we approximate the lower spectrum of the Lipkin Hamiltonian, we only consider a $N + 1$ dimensional subspace. While diagonalizing an $N + 1$ dimensional matrix is typically an easy task for a computer, the problem arises when N becomes very large, and in particular when one requires many evaluations on the parameter domain.

2.2. The Pairing model

Our main experimental testbed is the pairing model. Just like with the Lipkin model, there are many ways to formulate it, but we narrowed our use to one specific example, which we adapted from Hjorth-Jensen [4].

We consider a system of equally spaced doubly degenerate single particle levels, labeled by a level index p and spin projection $\sigma \in \{+, -\}$. The Hamiltonian is given by

$$H = H_0 + V,$$

where

$$H_0 = \delta \sum_{p,\sigma} (p - 1) a_{p\sigma}^\dagger a_{p\sigma},$$

and

$$V = -\frac{1}{2} g \sum_{p,q} a^\dagger_{p+} a^\dagger_{p-} a_{q-} a_{q+}.$$

Here δ denotes the single particle level spacing, g is the interaction strength and a and $a^\dagger$ are annihilation and creation operators respectively. In all experiments in this report we fix $\delta = 1$, meaning the model is only dependent on g. The interaction term V takes a pair of particles between levels. It acts only on pairs and preserves non-broken pairs. In this report, we restrict ourselves to the subspace with no broken pairs. Hence, our basis states are given by all configurations in which pairs occupy different single particle levels. If there are n_{levels} available pair levels and n_{pairs} occupied pairs, the Hilbert space is given by all combinations of pair occupations. Its dimension is therefore

$$\dim(\mathcal{H}) = \binom{n_{\text{levels}}}{n_{\text{pairs}}}.$$

2.3. Learning tasks

We now specify the quantities which we want to approximate. The learning problems in this report can be separated in two categories. Firstly, we have static spectral problems, where the model attempts to learn the low-lying eigenvalues of the Hamiltonian as functions over the parameter space. The second category is time-evolution problems, where the model tries to learn time-dependent quantities generated from the spectral properties of the Hamiltonian.

For both the Lipkin and pairing model we approximate the low energies of the Hamiltonian. If x denotes a parameter value, where x corresponds to V in the Lipkin model and g in the pairing model, we write its ordered eigenvalues as

$$\lambda_1(x) \leq \cdots \leq \lambda_n(x).$$

The static learning task is then to approximate the map

$$x \mapsto (\lambda_1(x), \ldots \lambda_k(x)),$$

for a fixed $k < n$. In other words, we want to learn how the lowest eigenvalues vary as the parameter changes. In the pairing model specifically we also study more complicated dynamic problems, where one first prepares an initial state $|\psi_0\rangle$, and then evolves it under the pairing Hamiltonian $H(g)$, meaning the time-evolution is given by

$$|\psi(t,g)\rangle = e^{-itH(g)} |\psi_0\rangle .$$

The first time-evolution quantity we look at is the *surival probability*

$$S(g,t) = |\langle \psi_0 | \psi(t,g) \rangle|^2 \in [0,1],$$

which Hopjan and Vidmar [7] explain as the squared overlap between $|\psi_0\rangle$ and its time-evolved state $e^{-itH} |\psi_0\rangle$, and is a standard quantity for studying quantum time-evolution. Essentially, the survival probability measures how much of the initial state remains after time-evolution. The closer it is to 1, the closer the time-evolution is to the initial state, and vice versa. The second time-evolution quantity we consider is a time-dependent observable expecation value of the form

$$\langle \psi(t,g) | O | \psi(t,g) \rangle,$$

where O is a fixed observable. In this report, O is a pair occupation observable, measuring whether pair level p is occupied. It is given by

$$O_p = n_{p+} n_{p-},$$

where $n_{p\sigma} = a^\dagger_{p\sigma} a_{p\sigma}$ is the number operator for the single particle state $p\sigma$. In our setting this is simply a diagonal operator with value 1 if level p is occupied and 0 otherwise. In the experiments, we use $p = 0$, corresponding to the lowest pair level. Hence, this observable gives the probability of finding a pair on the lowest pair level as time and pairing strength evolves.

3. PARAMETRIC MATRIX MODELS

In the previous section we introduced the quantities we wish to approximate, being the time-evolution spectrum of a Hamiltonian, as well as the aforementioned time-dependent quantities. We now describe the class of models used for approximating these quantities, namely parametric matrix models (PMMs).

PMMs are a family of machine learning algorithms, that has proven to be flexible in tackling different problems, as shown in Cook et al. [3]. In this report, we will discuss what is known as an affine eigenvalue PMM (AE-PMM), which, as we will soon see, is a very natural construction to approximate the spectrum of a Hamiltonian. The basic AE-PMM consists of a primary matrix P, which is typically a square Hermitian matrix dependent on trainable parameters θ, this matrix is diagonalized and produces eigenvalues as output. Given input features $c \in \mathbb{R}^p$. The primary matrix is an affine transformations on the form

$$P_\theta(c) := M_0(\theta) + \sum_{\ell=1}^{p} c_\ell M_\ell(\theta) \in \mathbb{C}^{n \times n},$$

where M_j are parametrized Hermitian square matrices whose elements are found by minimizing the mean squared error between the target values and the primary eigenvalues. The dimension n is a user chosen hyperparameter and serves as the PMM dimension, and is typically much smaller than the physical dimensions of the learning task. One of the remarkable properties of AE-PMMs is that, despite their simple form, they are highly

True Object	Surrogate Object
$H_0 \in \mathbb{C}^{d\times d}$	$M_0(\theta) \in \mathbb{C}^{n\times n}$
$H_1 \in \mathbb{C}^{d\times d}$	$M_1(\theta) \in \mathbb{C}^{n\times n}$
$H(x)$	$M_\theta(x)$
$f(x)$	$\hat{f}_\theta(x)$

Table I. An illustration of the true physical objects and the approximated counterparts provided by the PMM.

expressive. In fact as proved by [3], the following theorem holds.

Theorem 1 (Universal Approximation Theorem)**.** *Assume we have input features with a compact domain D and let $f : D \to \mathbb{R}$ be a continuous function. Then there is an AE-PMM that approximates f uniformly on D.*

This construction is especially natural for many body models, such as the Lipkin and pairing model. In both cases, the true Hamiltonian, $H \in \mathbb{C}^{d\times d}$ depends affinely on a physical parameter, and the PMM mirrors this structure with a trainable Hermitian surrogate. The resulting approximation is therefore not only low dimensional, but it also structurally aligns with the underlying physics. For the static eigenvalue task, the AE-PMM approximates the aforementioned map

$$x \mapsto (\lambda_1(x), \dots \lambda_k(x)) .$$

We denote the true map by $f(x)$ and the surrogate map by $\hat{f}(x)$. A schematic of the approximation can be seen in table I. The loss function we use during training is the standard mean square error given by

$$L(\theta) := \frac{1}{m}\sum_{i=1}^{m} \|\hat{f}_\theta(x_i) - f(x_i)\|^2.$$

For input features $\{(x_i, f(x_i))\}$ the training loop goes as follows:

Step 1) Initialize $M_0(\theta)$ and $M_1(\theta)$ randomly.

Step 2) For a chosen number of iterations or a stopping criterion:

- Compute predictions $\hat{f}(x_i)$.
- Compute the loss given by $L(M_0(\theta), M_1(\theta))$.
- Compute gradients $\nabla_{M_0} L$ and $\nabla_{M_1} L$.
- Update $M_0(\theta)$ and $M_1(\theta)$ with a chosen gradient descent method.

Step 3) Return the best parameters θ found.

How much the model is trained naturally depends on the number of epochs specified. The time-evolution tasks are more demanding than the static eigenvalue tasks, depending both on the interaction parameter and a time parameter. While we use the same basic AE-PMM construction as in the static eigenvalue case, the role of the learned primary matrix is different. It now serves as an effective Hamiltonian that tries to emulate the true time-evolution in the smaller n-dimensional space. Once the PMM has been trained, its eigenvalues and eigenvectors also determine a corresponding unitary evolution. To see this, assume $|v_1(x)\rangle, \dots, |v_n(x)\rangle$ is an orthonormal basis of eigenstates of the PMM primary $P_\theta(x)$, while $\lambda_1(x), \dots, \lambda_n(x)$ are the corresponding eigenvalues. Analogous with the exact time-evolution generated by the true Hamiltonian, the PMM time-evolution is defined by requiring that each eigenvector acquires the phase factor $e^{-it\lambda_j(x)}$. Thus, if a state is expanded in the PMM eigenbasis, its time-evolution is obtained by multiplying each spectral component by the corresponding phase. In the exact pairing problem, the initial state is taken to be a reference ground state prepared at some fixed parameter. The PMM mimics this structure, by taking the approximated state to be the ground state of the PMM primary, and then fixing it at the corresponding preparation parameter. From here the PMM surrogate generates the approximate time-evolution behavior, such as the survival probability.

As previously mentioned we also consider time-dependent observables of the form

$$\langle\psi(t,g)|O|\psi(t,g)\rangle.$$

This leads to two different approaches within the PMM framework. The first and maybe the most natural strategy is to let the PMM learn both the effective Hamiltonian as well as a trainable observable matrix, acting in the same space. The predicted time-evolution is then computed from the evolved state using the learned Hamiltonian and observable matrix. This is the more flexible of the two strategies, and allows the model more degrees of freedom to adapt both the time-evolution and the readout.

In the second strategy, only the effective Hamiltonian is learned, while the true observable is fixed. This naturally constrains the degrees of freedom the model is allowed for approximation, and therefore becomes a more challenging problem. Simply due to the model no longer being able to improve the fit by modifying the readout itself. Instead the primary matrix must generate the correct time-dependent behavior relative to the fixed observable, which reduces the models expressivity. For this reason, a good performance on the fixed observable setting gives stronger evidence that the PMM is capturing the underlying physical system, rather than just adapting its observable to the target data. While this can be considered a more valuable experiment, there is one structural difficulty with this strategy. Namely that the fixed observable lives in the full Hilbert space of the pairing model, whereas the PMM surrogate typically acts on a much smaller space, leading to a dimensional mismatch in the model readout.

As we will later see, this is a problem which we solve by introducing the compression framework.

4. THE COMPRESSION METHOD

As discussed at the end of the previous section, the compression method serves an important role in this report. For PMMs it has two distinct applications. Firstly, we use it directly on the surrogate Hamiltonian to further reduce dimensionality and thus for computational speedup. Secondly, it is used for solving the dimensional mismatch problem described earlier for the time-evolution learning task. In this report we present the necessary framework for applying the compression method to PMMs. The formulation of this method is adapted from and reuses the authors own work in Alsvik [8]. This source contains a more extensive construction of the compression method along with proofs of the claims made in this section.

4.1. The compression operator

Let $H(x) \in \mathbb{C}^{d\times d}$ be a Hermitian matrix depending on a parameter $x \in [a, b]$, and let

$$\lambda_1(x) \leq \cdots \leq \lambda_d(x)$$

denote its eigenvalues, with corresponding eigenvectors

$$|v_1(x)\rangle, \ldots, |v_d(x)\rangle.$$

In this report, this Hermitian matrix will be both the true Hamiltonian as well as the PMM surrogate. For a fixed integer $k < d$, we define the associated low-energy subspace by

$$\mathcal{L}(x) := \operatorname{span}\big(|v_1(x)\rangle, \ldots, |v_k(x)\rangle\big),$$

and let $P_k(x)$ denote the orthogonal projection onto this subspace. Since the low-energy subspace varies as x changes, the basic idea of the compression framework is to find a smaller subspace that captures the low-energy eigenspaces as well as possible over the parameter interval. To do this, we must first define the compression operator.

$$Q_k := \frac{1}{b-a}\int_a^b P_k(x)\, dx.$$

This operator averages the low-energy projections over the parameter domain. Essentially, directions that often appear in the low-energy eigenspaces get large weight in Q_k, and vice versa. As shown in Alsvik [8], the leading eigenspaces of Q_k gives an optimal reduced subspace in terms of an average over the parameter interval. More precisely, among all r-dimensional subspaces, the one spanned by the top r eigenvectors of Q_k maximizes the average overlap with the set of low-energy subspaces $\mathcal{L}(x)$. As a result of this, if $|u_1\rangle, \ldots, |u_r\rangle$ are the first r eigenvectors of Q_k, we can collect them in a matrix

$$B_r = \big[\, |u_1\rangle \cdots |u_r\rangle \,\big]$$

with orthonormal columns, that will define an optimal r-dimensional subspace used for compression. In practice, for our full Hermitian matrix $H(x)$, we define

$$H_r(x) := B_r^\dagger H(x) B_r \in \mathbb{C}^{r\times r},$$

which represents $H(x)$ on the reduced space. In this report, $H(x)$ is either the PMM surrogate, or the full Hamtiltonian. We go over each use case in the next sections.

4.2. Compression of the PMM surrogate

The first and perhaps most simple use case of the compression method is using it on the surrogate provided by the PMM. While the PMM already reduces the dimensionality of the problem, one can further reduce it with compression in order to gain computational speedups. To do this, one must choose an r that sufficiently approximates the surrogate. A natural way to do this, is to set some tolerance ϵ for the error $e_{k,r}$ that arises from compression, and set the optimal r as

$$r^*(\epsilon) := \inf\{r \in \mathbb{N} \,:\, e_{k,r} \leq \epsilon\}.$$

We define the error by

$$\delta_{k,r}(x) := \max_{1\leq j\leq k} (\lambda_j^{(r)}(x) - \lambda_j(x)),$$

where $\lambda_j^{(r)}$ denotes the eigenvalues of the compressed $H_r(x)$. We then define the total compression error as

$$e_{k,r} := \sup_{x\in[a,b]} \delta_{k,r}(x).$$

Meaning, this error is a metric of the worst case error between the compressed energy and the time-evolution eigenvalues of the PMM. By using the Courant-Fischer-Weyl minmax principle in the form of Corollary III.1.2 in Bhatia [9], one can show that the error is variational, in the sense that our compressed eigenvalues will not be smaller than the true eigenvalues. This ensures that the error is well-defined, in the sense that if we find an r that is below the tolerance, we will not suddenly go above it by increasing r. Moreover, if ϵ_{tot} denotes the total error between the true low-energy eigenvalues of $H(x)$ and the compressed PMM eigenvalues, and ϵ_{sur} denotes the surrogate error, then one can show that

$$\epsilon_{\text{tot}} \leq \epsilon_{\text{sur}} + \epsilon,$$

where ϵ is some pre defined compression error tolerance. Meaning that the compression error tolerance can be seen as an additional hyperparameter in the PMM model, which specifies the error you are willing to pay for compression.

4.3. Direct compression of the exact model

The second way we apply compression in this report is to find a reduced exact model, directly from the full Hamiltonian. This is what allows us to the solve the dimensional mismatch problem, that was mentioned when we discussed the time-evolution learning task. As mentioned earlier, by constructing the compression operator, we find the reduced Hamiltonian by

$$H_r(x) := B_r^\dagger H(x) B_r \in \mathbb{C}^{r\times r}.$$

The crucial question is how much of the physical structure the compression preserves. In particular, for the time-evolution setting, where the target quantities depend not only on the Hamiltonian, but also on the initial state and a chosen observable. In order to apply our PMM framework, we must compress all of these ingredients as well, not only the Hamiltonian. If $|\psi_0\rangle$ denotes the initial state in the full state space, we define its reduced counterpart by projecting it onto the compressed subspace, and then renormalizing. Giving us that the compressed initial state becomes

$$|\psi_{0,r}\rangle := \frac{B_r^\dagger |\psi_0\rangle}{\| B_r^\dagger |\psi_0\rangle \|}.$$

Likewise, if O is the observable of interest on the full state space, its compressed version is defined by

$$O_r := B_r^\dagger O B_r.$$

Using these objects, we are able to compute the time-evolution quantities entirely within the compressed r-dimensional space. This is what allows us to compute the reduced survival probabilities and reduced observable expectation values, in a way that is compatible with the PMM surrogate. In this setting $r = n$, so that we match the PMM dimension. As we will see in the results section, the compressed quantities remain close to the corresponding full exact benchmarks.

To summarize, compression of the exact model is used in two ways. Firstly, it provides compressed "exact" survival probability and observable benchmarks. Secondly, it provides the fixed reduced observables used in the PMM time-evolution experiments, allowing us to solve the problem of mismatching dimensions when applying PMMs for time-evolution tasks.

5. IMPLEMENTATION

This section presents the numerical framework, training setup for the PMMs, baseline models and the evaluation methods used in this project. The Github repository that includes the code that generated all the results used in this report, can be found at Alsvik [10].

5.1. Software and computational environment

All the numerical experiments in this report were performed using Python. For linear algebra and spectral computations, we used NumPy and SciPy. For optimization of the PMMs and implementing neural network baselines, we used PyTorch. Figures were generated with matplotlib, while the heatmaps were generated using the seaborn package. All computations and in particular runtime comparisons were performed locally on a MacBook Pro with an Apple M4 pro chip and 24 GB of unified memory.

5.2. Target data generation and model setup

All the target data in this report is generated directly from the Hamiltonians, by exact numerical diagonalization in Python. Thus, both the PMMs and the neural network baselines are always trained against the exact solutions from the physical models. Our general testing philosophy is first running baseline tests, and thereafter moving to more demanding problems.

For the Lipkin model, we consider a smaller system with $N = 20$ with $V \in [0, 2]$ as a baseline, while the main experiments are performed for $N = 200$, using the intervals $V \in [0, 2]$ and $V \in [0, 4]$, giving a Hilbert space dimension of 201. In all the Lipkin experiments, the learning targets consists of the lowest $k = 3$ eigenvalues of the Hamiltonian.

For the pairing model, our main testbed is in the $pnum = hnum = 10$ configuration. Giving a Hilbert space dimension of

$$\binom{10}{5} = 252.$$

The interaction strength is varied over the intervals $g \in [0, 2]$ and $g \in [0, 4]$. As in the Lipkin case, we primarily look at the lowest $k = 3$ eigenvalues obtained from exact diagonalization. The same framework is used to generate the time-evolution data, although for runtime purposes, we considered a smaller system with $pnum = hnum = 8$, giving a Hilbert space dimension of

$$\binom{8}{4} = 70.$$

We prepared the initial state as the exact ground state, at a fixed preparation parameter $g_{\text{prep}} = 0.5$. For the fixed pair-occupation observable O, we mainly study the occupation at the lowest pair level.

5.3. PMM implementation and training

In all the experiments performed in this report the Hamiltonians are real symmetric, hence we restricted

the PMM to be real valued as well. This reduced the number of trainable parameters, along with simplifying the numerical implementation. We also enforced symmetry of the primary matrices during the forward pass.

For the static eigenvalue experiments, the PMM input consists of a single parameter, being the interaction strength. While the target is the block of the lowest $k = 3$ eigenvalues of the exact Hamiltonian. In the Lipkin baseline experiments, we used 20 training points and 100 test points. While in the larger Lipkin and Pairing experiments, we used 40 training points and 200 test points on the intervals considered in each experiment. We chose a much denser test grid than training grid, to make sure that we got an accurate evaluation of how the model generalized between training points, instead of letting the model fit to the sampled training points. The PMM was trained by minimizing the mean squared error between the predicted and exact eigenvalue blocks. For the time-evolution experiments, the PMM input consists of the interaction parameter, together with the time variable. The interaction and time were sampled on a 24×32 training grid, and a 48×80 test grid in (g, t). For the same reason as in the static setting, the test grid was denser than the training grid.

The PMMs were trained with the Adam optimizer from PyTorch, with learning rate 10^{-2}. The number of epochs depended on the experiment, ranging between 4000 and 20000. In each case, the trainable matrices were initialized randomly at small scale. All input variables were linearly normalized before being passed to the model. The PMM dimension n was treated as one of the main architectural hyperparameters throughout the report. For all benchmarks, several dimensions were tested, their range depending on the difficulty of the problem.

Since the static and time-evolution tasks share the same overall optimization philosophy, but differ in their forward maps, we summarize the two training procedures separately below. Algorithm 1 gives the generic training loop for the static affine eigenvalue PMM, while Algorithm 2 shows the corresponding procedure for the time-evolution setting.

Algorithm 1 Training of an affine eigenvalue PMM

Require: training set $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{m}$, dimension n, retained eigenvalue count k, learning rate η, number of epochs T, initial matrices $M_0, \dots, M_p$
Ensure: best trained parameters θ^*

$L_{\text{best}} \leftarrow \infty$
$\theta^* \leftarrow (M_0, \dots, M_p)$
for $\tau = 1, \dots, T$ **do**
  for $\ell = 0, \dots, p$ **do**
    Symmetrize $M_\ell \leftarrow \frac{1}{2}(M_\ell + M_\ell^\top)$
  end for
  for each $(x_i, y_i) \in \mathcal{D}_{\text{train}}$ **do**
    Construct the primary $P_\theta(x_i) = M_0 + \sum_{\ell=1}^{p}(x_i)_\ell M_\ell$
    Compute eigenvalues $\mu_1(x_i) \le \cdots \le \mu_n(x_i)$ of $P_\theta(x_i)$
    Form prediction $\hat{y}_i \leftarrow (\mu_1(x_i), \dots, \mu_k(x_i))$
  end for
  Compute loss $L(\theta) \leftarrow \frac{1}{m}\sum_{i=1}^{m} \|\hat{y}_i - y_i\|^2$
  Compute $\nabla_\theta L(\theta)$ by backpropagation
  Update θ using Adam with learning rate η
  if $L(\theta) < L_{\text{best}}$ **then**
    $L_{\text{best}} \leftarrow L(\theta)$
    $\theta^* \leftarrow \theta$
  end if
end for
return θ^*

Algorithm 2 Training of a time-evolution PMM

Require: training set $-\mathcal{D}_{\text{train}} = \{((x_i, t_i), y_i)\}_{i=1}^{m}$, preparation parameter x_{prep}, dimension n, learning rate η, number of epochs T, initial PMM parameters θ
Ensure: best trained parameters θ^*

$L_{\text{best}} \leftarrow \infty$
$\theta^* \leftarrow \theta$
for $\tau = 1, \dots, T$ **do**
  Symmetrize the trainable matrices appearing in θ
  Construct the PMM primary at x_{prep} and compute its ground state $|\phi_{0,\theta}\rangle$
  for each $((x_i, t_i), y_i) \in \mathcal{D}_{\text{train}}$ **do**
    Construct the PMM primary at x_i
    Compute its eigenpairs $\{(\lambda_j(x_i), |v_j(x_i)\rangle)\}_{j=1}^{n}$
    Expand $|\phi_{0,\theta}\rangle$ in the PMM eigenbasis
    Compute the timeevolved state $|\phi_\theta(t_i; x_i)\rangle$
    Compute the task dependent prediction $\hat{y}_i$
  end for
  Compute loss $L(\theta) \leftarrow \frac{1}{m}\sum_{i=1}^{m} |\hat{y}_i - y_i|^2$
  Compute $\nabla_\theta L(\theta)$ by backpropagation
  Update θ using Adam with learning rate η
  if $L(\theta) < L_{\text{best}}$ **then**
    $L_{\text{best}} \leftarrow L(\theta)$
    $\theta^* \leftarrow \theta$
  end if
end for
return θ^*

5.4. Testing philosophy and benchmark models

The main testing philosophy in this report is to evaluate the PMMs against exact quantities, whenever these are available. While this is a useful comparison, it is not sufficient to establish PMMs as a viable machine learning method. Hence, we also compare with more standard machine learning methods, being neural networks.

For the static spectral tasks, we compare the PMMs with feedforward neural networks trained on the same input data as the PMMs. We implemented the networks in PyTorch and the specific configuration used was selected by grid search over on to three hidden layers with hidden layer sizes $32, 64, 128$. We also tried several

activation functions such as ReLU and tanh, as well as regularization settings. We selected the best performing model to use in our benchmarks. For the time-evolution tasks, we use gated recurrent unit models (GRU) as baselines, since these are well suited for sequential data, such as time-evolution points. They are also more robust against the vanishing gradient problem than standard recurrent neural networks, as described by Cho et al. [11]. The GRU models were implemented using PyTorch, with ReLU activation functions. Similarly to standard neural network in the static tasks, their hyperparameters were selected with a grid search, after which the best performing model was chosen as the benchmark. As these are standard machine learning methods, we do not explain the theory behind these methods. For a detailed treatment of neural network variants see Raschka and Mirjalili [12].

5.5. Numerical implementation of the compression method

The compression operator was approximated numerically by replacing the parameter integral with a finite average over a uniform parameter grid, giving us that

$$Q_k \approx \frac{1}{m} \sum_{j=1}^{m} P_k(x_j),$$

where $x_1, \ldots, x_m$ are grid points in the parameter interval and $P_k(x_j)$ denotes the orthogonal projector onto the span of the lowest k eigenvectors at the point x_j. In all experiments, these projectors were obtained by numerical diagonalization of the corresponding PMM surrogate or Hamiltonian at each grid point. The compression error $e_{k,r}$ is always computed on a separate grid than for constructing Q_k, so that we keep the diagnostics and the basis construction numerically distinguished. Once Q_k has been constructed, we apply it as described in the previous sections.

When compression is applied to a trained PMM surrogate, the reduced dimension r is chosen by setting a tolerance on the compression error, and using the lowest r that satisfies the error tolerance level. In practice, we implemented this search numerically by doing a coarse grid search over a set of candidate dimensions. Once a dimension satisfying the tolerance level is found, we search in the neighboring interval in order to find the smallest valid rank. In the direct compression time-evolution experiments, the role of compression is different. There, we aim to solve the dimensional mismatch problem, hence the reduced dimension r is always set to be the same as the PMM dimension n, meaning $r = n$. Meaning it is not necessary to perform a grid search in this setting.

5.6. Error and comparison metrics

Several different error measures were used in the experiments, depending on the task and the type of comparison being made. These broadly falls into three categories, being test error, compression related errors and model comparison diagnostics. We briefly go over each category.

For test accuracy on unseen data, we use the test mean squared error, together with the mean absolute error, maximum absolute error and root mean square error (RMSE). In the static spectral experiments, these quantities are computed over the full eigenvalue block across the test grid. In the time-evolution experiments, the same metrics are computed over the corresponding (g, t)-grid for survival probabilities and observable expectation values. The main compression specific diagnostic used is the beforementioned compression error $e_{k,r}$. This quantity should be interpreted as the additional error resulting from compression, on top of the approximation error from the PMM surrogate. Lastly, we also compute model comparison quantities, such as parameter count, training time and when relevant, evaluation time over the parameter interval grid.

It is important to distinguish the reference objects used in the different experiments. Depending on the setting, a model may be compared against the full exact model, a trained PMM surrogate, a compressed PMM surrogate, or a compressed benchmark of matched dimension. One must therefore be careful in interpreting the error metrics, relative to the specific benchmark used in that experiment.

5.7. Runtime measurments and speedup analysis

As mentioned, we compare models on runtime quantities. Since one of our motivations for PMMs and compression is to reduce computational expense, it is important to distinguish between one time preprocessing costs and repeated evaluation costs. For the compressed methods, preprocessing includes the numerical construction of the compression operator Q_k, the diagonalization of Q_k, as well as the formation of the reduced basis and compressed model. Once this has been done, the reduced model can be evaluated repeatedly on a parameter grid at a lower cost than the corresponding full model. We therefore measure both the preprocessing time and the scan time of the reduced model. In this report, a scan means one complete evaluation of the relevant quantity, over the whole parameter interval grid. The corresponding exact scan time is measured by evaluating the full model on the same grid.

We also consider the break even point for the compression method, meaning the number of repeated

scans required before the upfront preprocessing cost of compression is "worth it", in terms of the lower scan time of the reduced model. The break even scan count is computed after the PMM has already been trained, and is computed in the following way.

Let T_{PMM} be the time required to perform one full PMM scan, and let T_{red} be the time required to perform the same scan using the compressed PMM basis. We denote the compression preprocessing time by T_{prep}. The compressed method becomes advantageous once the total scan time savings becomes higher than the preprocessing cost. Thus the break even number of scans is

$$N_{\mathrm{break}} = \frac{T_{\mathrm{prep}}}{T_{\mathrm{PMM}} - T_{\mathrm{red}}}.$$

5.8. Reproducibility and repository structure

All experiments in this report were carried out with fixed seeds and fixed numerical settings within each configuration. The main hyperparameters, parameter intervals, grid sizes and training settings are either specified in the implementation section or together with the corresponding experiments. For several experiments when runtime permitted it, we used multiple seeds in order to get a better overview of the PMMs performance. In these experiments, we generally picked the best performing seed for each method, to compare the methods at their "peak performance". It would also be useful to test the methods by taking the average over seeds, but due to time constrains, this was not included in the report. All numerical results for each seed are available in the Github repository. To keep the report readable, we include only the main methodological choices and the most informative numerical results. Additional exploratory experiments, such as benchmark grid searches and training budget experiments, can be found in the Github repository. We will also refer to numerical results that can be found in the repository. The repository is self contained and includes all source files needed to reproduce the results in this report. It also contains the required dependency information, together with a short usage guide for running the experiments.

6. RESULTS AND DISCUSSION

We now present our main results and discuss their implications. We first look at the static learning tasks for the Lipkin model, where the PMM is compared against exact diagonalization and a neural network baseline. We then move on to the pairing model, first for the static tasks, before moving on to the time-evolution experiments. For all the results given in time, the unit is seconds. Moreover, the compression error tolerance is 0.01 for all experiments in this report, except for in the time-evolution experiments, where the compressed dimension is always fixed to be the same as the PMM dimension.

6.1. Results from the Lipkin model experiments

The Lipkin model serves as our first static learning task. We initially tested the PMM on a smaller Lipkin model, as a "proof of concept", which can be seen in figure 1.

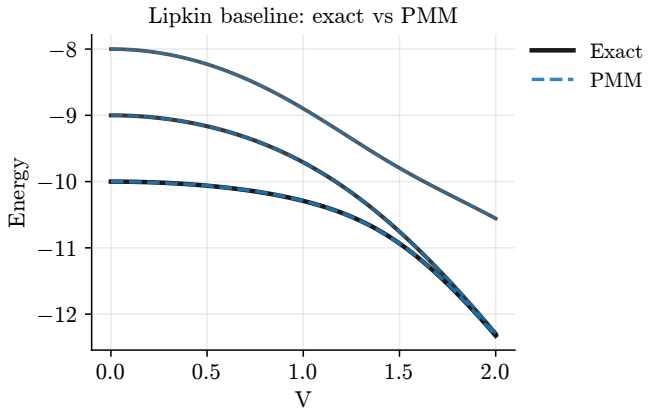

Figure 1. Baseline static Lipkin model experiment, with $N = 20$, $k = 3$, $V \in [0, 2]$ and PMM dimension $n = 10$.

The PMM approximates the low-lying spectrum very well, with a mean absolute test error of only 0.000984. We next consider a larger Lipkin experiment. Figure 2 compares the PMM with a feedforward neural network baseline.

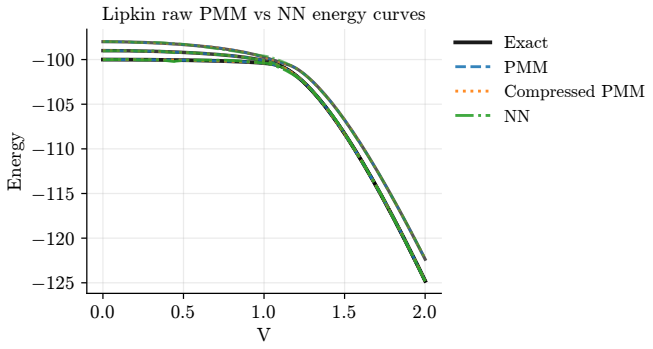

Figure 2. Comparison between the PMM, compressed PMM, exact spectrum and neural network baseline for the larger Lipkin experiment with $N = 200$ and PMM dimension $n = 96$.

While neither method performs particularly strong, the PMM gives a visibly better approximation than the neural network, where exact numerical values can be found in table II. However, the PMM has a dramatically higher training time as well as more trainable parameters than the neural network, being a limitation of the PMM in this setting. During model selection, we tested several different dimensions for the PMM in this setting, each being quite large. During testing, we found that using lower dimensions quickly deteriorated the approximation.

As we will soon see, the PMM dimensions required for the pairing model were smaller, which suggests that the Lipkin model is more demanding than the pairing model in this setting. A possible explanation could be that the low-lying Lipkin spectrum changes more sharply over the parameter interval as the interaction strength increases, increasing the need for expressivity in the approximation.

Model	Mean AE	Max AE	RMSE	Params	Train/setup
PMM	1.35×10^{-2}	5.20×10^{-2}	1.77×10^{-2}	18432	8118.02
NN	4.27×10^{-2}	2.30×10^{-1}	6.07×10^{-2}	17155	18.40

Table II. Lipkin PMM and neural network comparison for the larger static spectral learning task, with $N = 200$ and PMM dimension $n = 96$. The errors are computed on the test grid for the lowest three eigenvalues. The timing column includes the training and model-selection procedure used to obtain the reported model, and should not be confused with the compression preprocessing time used later.

6.2. Static pairing model results

While the results from the Lipkin model experiments were not an overwhelming success for the PMMs, the results for the pairing model are substantially stronger. As can be seen in figure 3 and table III, the PMMs achieve lower test errors than the neural network baseline, while also using much fewer trainable parameters. The training time is also much more comparable in the pairing setting, particularly for the 16-dimensional PMM, making the reward of lower test error and fewer parameters much more enticing.

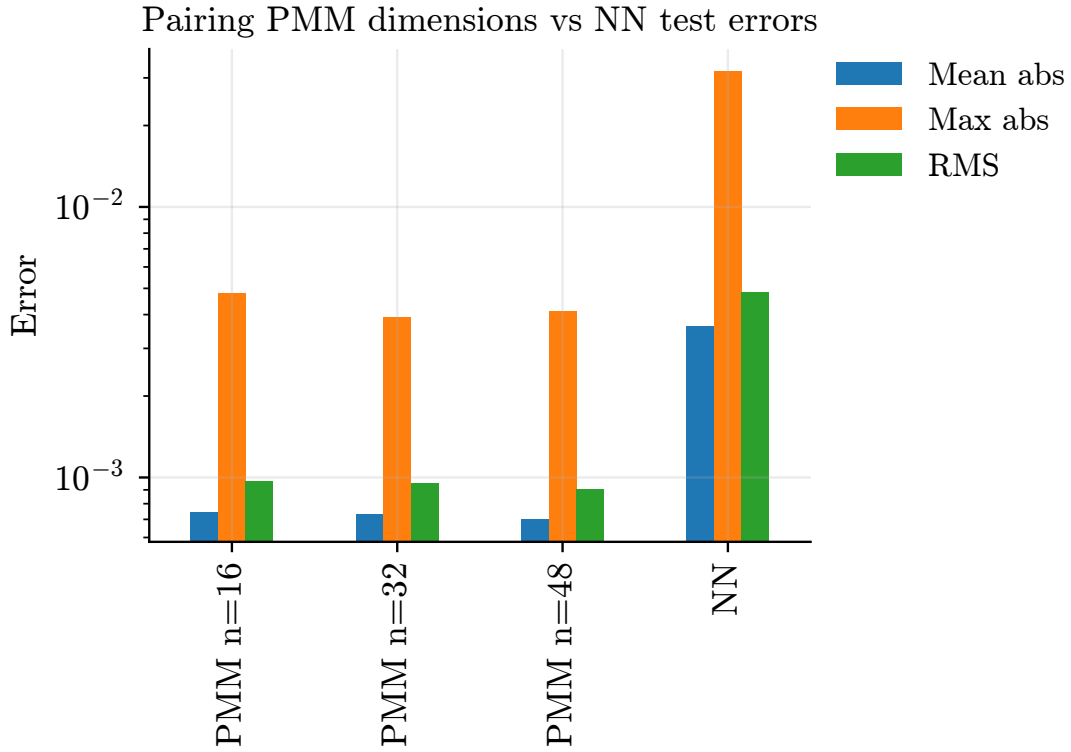

Figure 3. Low-energy eigenvalues test errors for PMMs of dimensions $n = 16, 32, 48$ for the $pnum = hnum = 10$ pairing model, compared with the neural network baseline. The errors are computed on the test grid for the lowest three eigenvalues. All PMM dimensions give substantially lower mean, maximum and RMS errors than the neural network, while using fewer trainable parameters.

Model	Dim.	Mean AE	Max AE	RMSE	Params	Train/setup
PMM	16	7.42×10^{-4}	4.78×10^{-3}	9.67×10^{-4}	512	36.77
PMM	32	7.30×10^{-4}	3.91×10^{-3}	9.50×10^{-4}	2048	124.16
PMM	48	7.00×10^{-4}	4.10×10^{-3}	9.02×10^{-4}	4608	294.31
NN	–	3.63×10^{-3}	3.17×10^{-2}	4.85×10^{-3}	17155	20.25

Table III. Static pairing model comparison between PMMs and the neural-network baseline. The errors are computed on the test grid for the lowest three eigenvalues. For the neural netowrk, the timing column includes the training and model selection procedure used for the comparison.

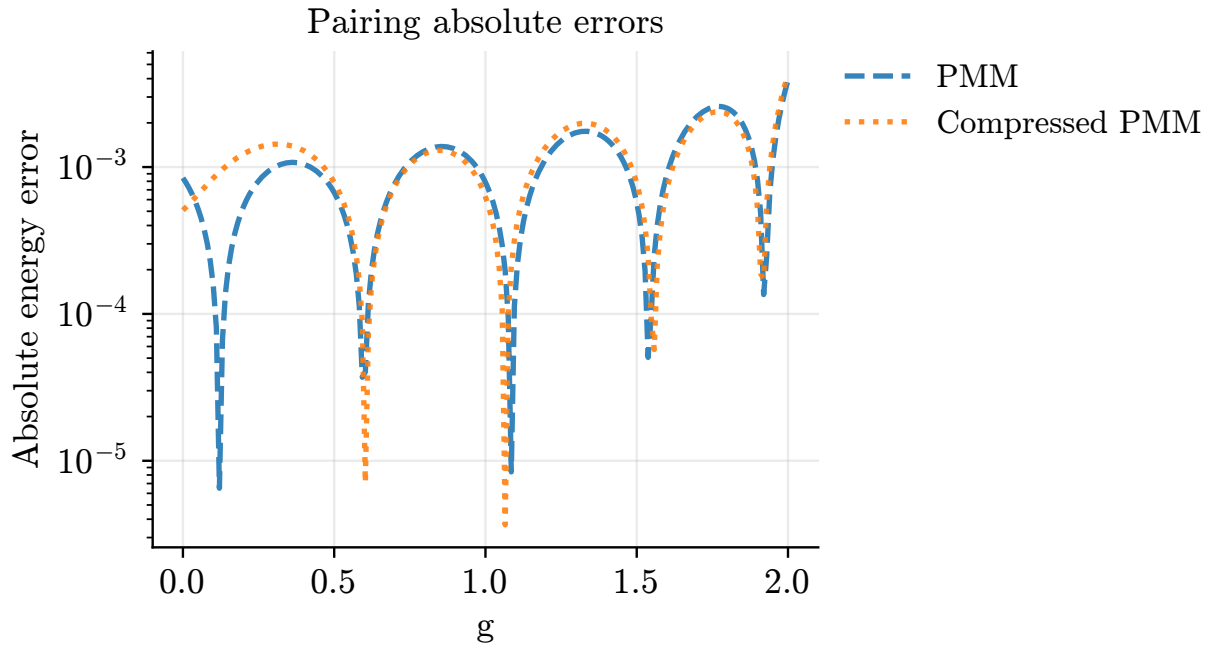

Figure 4. Absolute errors for the static pairing PMM regime sweep, comparing the uncompressed PMM with the compressed PMM over the interaction strength g. The compressed PMM stays close to the uncompressed PMM, indicating that compression preserves the low-energy behavior of the trained PMM surrogate. This experiment uses the $pnum = hnum = 10$ pairing model with PMM dimension $n = 32$, and compressed dimension $r^* = 14$.

Regime	n	r^*	e_{k,r^*}	$\mathrm{MAE}_{\mathrm{PMM}}$	$\mathrm{MAE}_{\mathrm{red}}$	T_{PMM}	T_{red}	T_{pre}	Speedup	N_{break}
$g \in [0, 2]$	16	11.67	3.47×10^{-3}	8.46×10^{-4}	9.79×10^{-4}	1.71×10^{-3}	1.51×10^{-3}	1.08×10^{-2}	1.13	66.14
$g \in [0, 2]$	32	12.67	3.98×10^{-3}	8.71×10^{-4}	9.73×10^{-4}	5.38×10^{-3}	1.93×10^{-3}	2.01×10^{-2}	2.80	5.84
$g \in [0, 2]$	48	12.33	6.67×10^{-3}	8.61×10^{-4}	1.29×10^{-3}	1.22×10^{-2}	2.24×10^{-3}	3.13×10^{-2}	5.47	3.15
$g \in [0, 4]$	16	12.67	7.77×10^{-4}	9.80×10^{-4}	9.75×10^{-4}	1.70×10^{-3}	1.70×10^{-3}	1.24×10^{-2}	1.00	–
$g \in [0, 4]$	32	12.67	6.15×10^{-3}	8.86×10^{-4}	1.21×10^{-3}	5.34×10^{-3}	1.93×10^{-3}	1.91×10^{-2}	2.77	5.65
$g \in [0, 4]$	48	15.33	4.70×10^{-3}	1.02×10^{-3}	1.15×10^{-3}	1.29×10^{-2}	2.75×10^{-3}	3.58×10^{-2}	4.67	3.55

Table IV. Compression and runtime summary for the static pairing PMM regime sweep with $pnum = hnum = 10$ and PMM dimensions $n = 16, 32, 48$. Here $\mathrm{MAE}_{\mathrm{PMM}}$ and $\mathrm{MAE}_{\mathrm{red}}$ are the mean absolute test errors of the original and compressed PMM, respectively. T_{PMM} is the scan time of the uncompressed PMM, T_{red} is the scan time of the compressed PMM, and T_{pre} is the one time compression preprocessing time. The reduced dimension r^* is chosen according to the compression error tolerance. The break even count N_{break} measures how many repeated scans are needed before the compression preprocessing cost is worth it. It does not include PMM training time. For $g \in [0, 4]$, $n = 16$, the timing difference was too small to give a finite useful break even number. All entries are averaged over the tested seeds, so r^* denotes the mean selected reduced dimension.

Apart from comparing the PMM with a neural network, we also performed a broader regime sweep with different PMM settings and seeds, together with compression. As can be seen in figure 4 and table IV, the compressed PMM remains close to the uncompressed PMM as well as the exact low-lying spectrum. We see a small local degradation for the smaller value of g, which looks somewhat dramatic with the log-axis, although the difference in mean absolute error is small. The main observation from table IV is that the PMMs can often be compressed to a significantly smaller dimension without adding much error. For instance, the $n = 32$ and $n = 48$ PMMs are reduced to dimensions around $r^* \approx 12$–15, while still retaining small compression errors. This suggests that even though the original PMM dimension is useful during training, the final trained surrogate contains some redundant directions that can be removed afterwards.

The speedup results also show that compression is most useful for the larger PMMs, which is not surprising. For $n = 32$ and $n = 48$, the compressed scans are several times faster than the uncompressed scans, and only a few compressed scans are required to break even.

On the other hand, for the $n = 16$ PMM, the benefit is much smaller. In one case, the compressed scan is even slightly slower than the uncompressed scan, despite the reduced dimension. A possible explanation is that the compressed matrices are dense, while the original PMM computation may still benefit from structure and small matrix size. Hence, for very small PMMs, the computational cost of working with the compressed representation can outweigh benefit from the dimensional reduction. Compression is therefore most useful once the original surrogate is large enough that the reduction in dimension dominates this cost. It is important to note that the neither PMMs nor compression remove the need for exact calculations entirely. The model still requires exact training data, which in this report is obtained by exact diagonalization. The approach is most useful in settings where many repeated evaluations are needed after training. This also explains why training time and break even analysis must be interpreted carefully. The PMM training cost is an offline cost, while the scan time savings matter only when the trained model is reused several times.

6.3. Time-evolution results

We now move on to the time-evolution experiments for the pairing model, where all experiments are done in the $pnum = hnum = 8$ pairing configuration. This is where the compression framework plays its second role in this report. Instead of compressing an already trained PMM surrogate, we now use compression directly on the exact Hamiltonian in order to construct reduced exact benchmarks. This allows us to compare the PMM with an exact model of the same effective dimension, and it also makes the fixed observable experiments in the next section possible.

We begin with the survival probability results. As described earlier, the initial state is prepared as the exact ground state at $g_{\text{prep}} = 0.5$, and is then evolved by using the Hamiltonian at the given value of g. The task is therefore to learn the map

$$(g, t) \mapsto S(g, t).$$

Figure 5 shows the absolute errors for the PMMs, the compressed benchmarks and the GRU baseline.

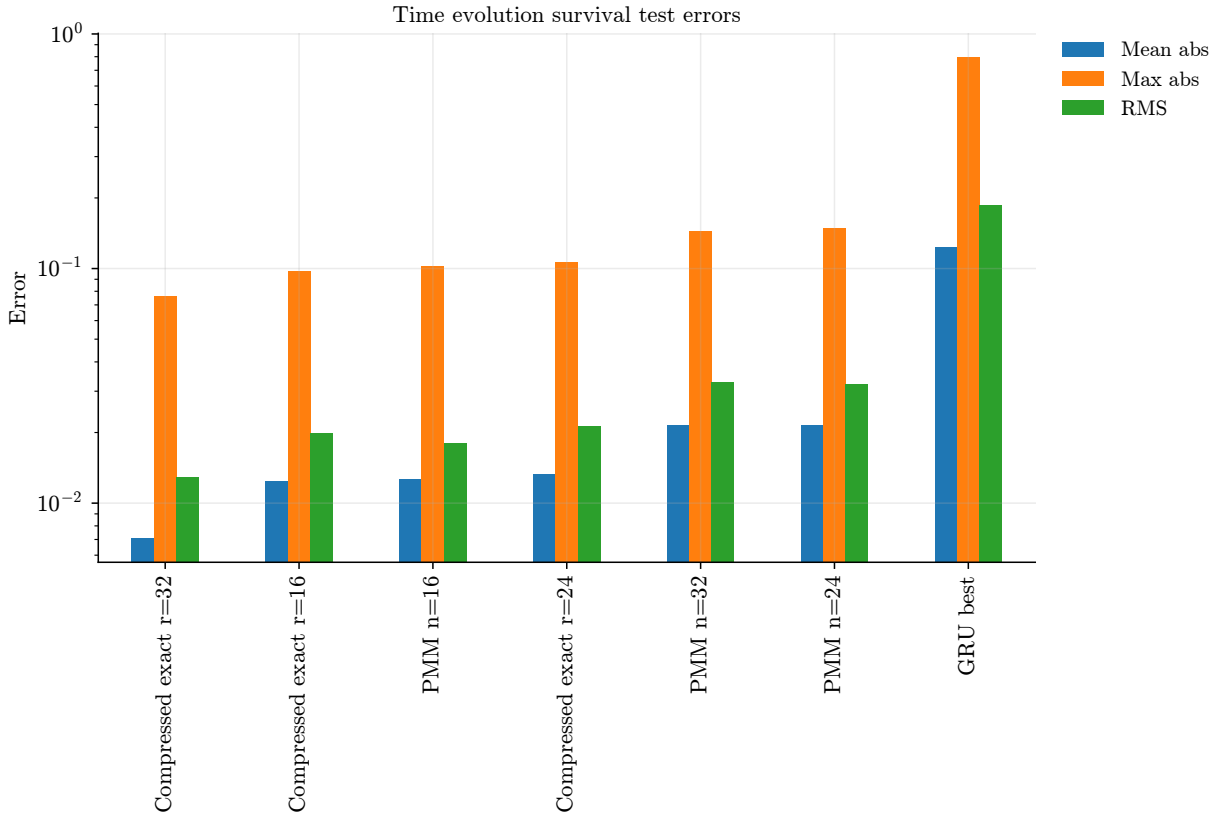

Figure 5. Survival probability absolute error for PMMs with dimensions $n = 16, 24, 32$, and compressed models of ranks $r = 16, 24, 32$ and with the best GRU baseline. Errors are computed against the full exact survival probability on the 48×80 test grid in (g, t).

Model	Dim.	Mean AE	Max AE	RMSE	Params	Train time
Compressed	16	1.24×10^{-2}	9.77×10^{-2}	1.99×10^{-2}	–	–
Compressed	24	1.33×10^{-2}	1.07×10^{-1}	2.12×10^{-2}	–	–
Compressed	32	7.09×10^{-3}	7.59×10^{-2}	1.29×10^{-2}	–	–
PMM	16	1.28×10^{-2}	1.00×10^{-1}	1.82×10^{-2}	512	52.88
PMM	24	2.15×10^{-2}	1.49×10^{-1}	3.23×10^{-2}	1152	101.34
PMM	32	2.15×10^{-2}	1.44×10^{-1}	3.27×10^{-2}	2048	168.85
GRU	–	1.23×10^{-1}	7.95×10^{-1}	1.87×10^{-1}	50817	87.86

Table V. Survival probability error metrics for the pairing time-evolution task, with PMM dimensions $n = 16, 24, 32$, and compressed benchmark dimensions $r = 16, 24, 32$. All errors are computed against the full exact survival probability on the 48×80 test grid. In this table, model selection is not a part of the time column. The best performing GRU over a grid search was chosen as the representative.

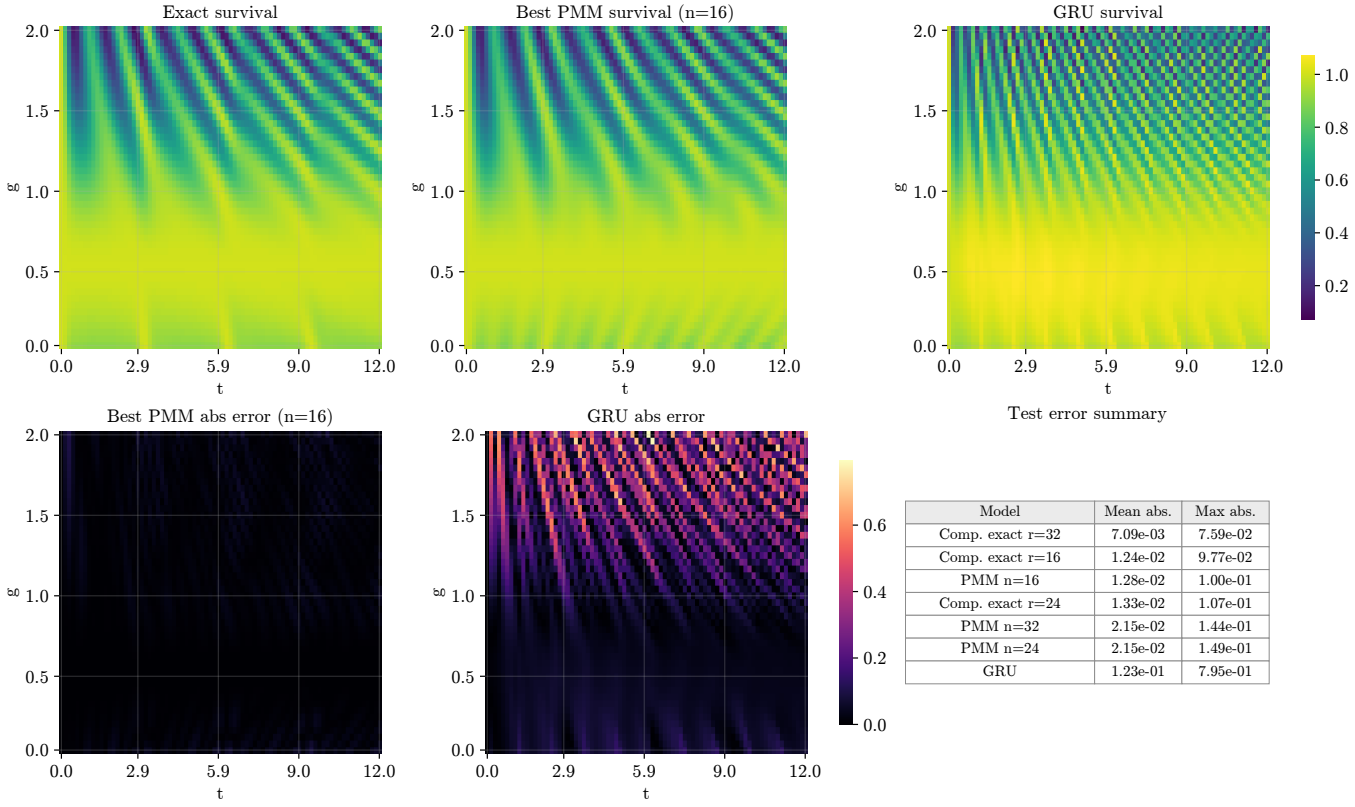

Figure 6. Error heatmaps for the survival probability task over the full (g, t) test grid. The figure compares the compressed benchmarks, the PMM predictions and the GRU baseline. The compressed models use ranks $r = 16, 24, 32$, with PMM dimensions $n = 16, 24, 32$. Only the best performing PMM model, with $n = 16$, is shown in the heatmaps. The color scale shows absolute error relative to the full exact survival probability. We include a small table repeating the mean absolute error and the max absolute error, for interpretability of the heatmaps.

We see that the PMM solidly outperforms the GRU baseline. We see that the $n = 16$ PMM is very close to the corresponding $r = 16$ compressed benchmark. This implies that the error relative to the full exact survival probability mostly stems from the compression error and not the PMM approximation. As seen in Table V, the PMMs reduce the error by around one order of magniture compared with the GRU baseline. We also see that that for the $n = 16$ PMM, the training time is actually lower than for the GRU, while for the larger PMMs the training time is somewhat longer. Across the board, the GRU uses a vast amount of trainable parameters, while the PMM uses relatively few, and in the $n = 16$ case, the difference is roughly two orders of magnitude. Interestingly the $n = 16$ PMM performed better than the larger models. In theory, a larger dimension should imply more expressivity, so this is probably a consequence of the larger PMMs being harder to optimize, and may require a higher training budget than 4000 epochs. It could also be due to numerical fluctuations on the particular seed used. Due to runtime concerns, this experiment was only performed on a single

seed with a smaller training budget. For a slightly more representative result, it could be beneficial to increase the training budget as well as using more seeds.

The heatmaps in figure 6 give a more detailed view of the error over the full (g, t) domain. The PMM error remains small over most of the test grid, while the GRU error is much larger over the majority of the test grid, which is particularly noticeable for the heatmap showing the absolute error for the 16-dimensional PMM, where the heatmap is close to being uni colored. Moreover, we see that the survival probability pattern is almost identical in the heatmaps for the exact survival probability and the 16-dimensional PMM. While the GRU visibly struggles to recreate the pattern. Since our test grid is denser than the training grid, the result indicates that the PMM generalizes better across the parameter domain, rather than only fitting the training points.

We now move on to a stricter test, where the target is a fixed observable. In this experiment, we want to approximate time-dependent pair-occupation values of the form

$$(g, t) \mapsto \langle \psi(t, g) | O | \psi(t, g) \rangle.$$

This task is more constrained than the survival probability experiment, because the output now depends on both the learned time-evolution and the observable used to measure the evolved state. As before, the full pairing model has $pnum = hnum = 8$, giving a Hilbert space dimension of 70, and the initial state is prepared as the exact ground state at $g_{\text{prep}} = 0.5$. We compare the two different PMM variants that we discussed earlier. The first one learns both the Hamiltonian as well as the observable readout. In the second variant we, we use compressed observable as a fixed readout. As previously mentioned, this is a more constrained problem since we remove degrees of freedom available to the PMM, in the sense that we do not let the model simply adapt to the readouts.

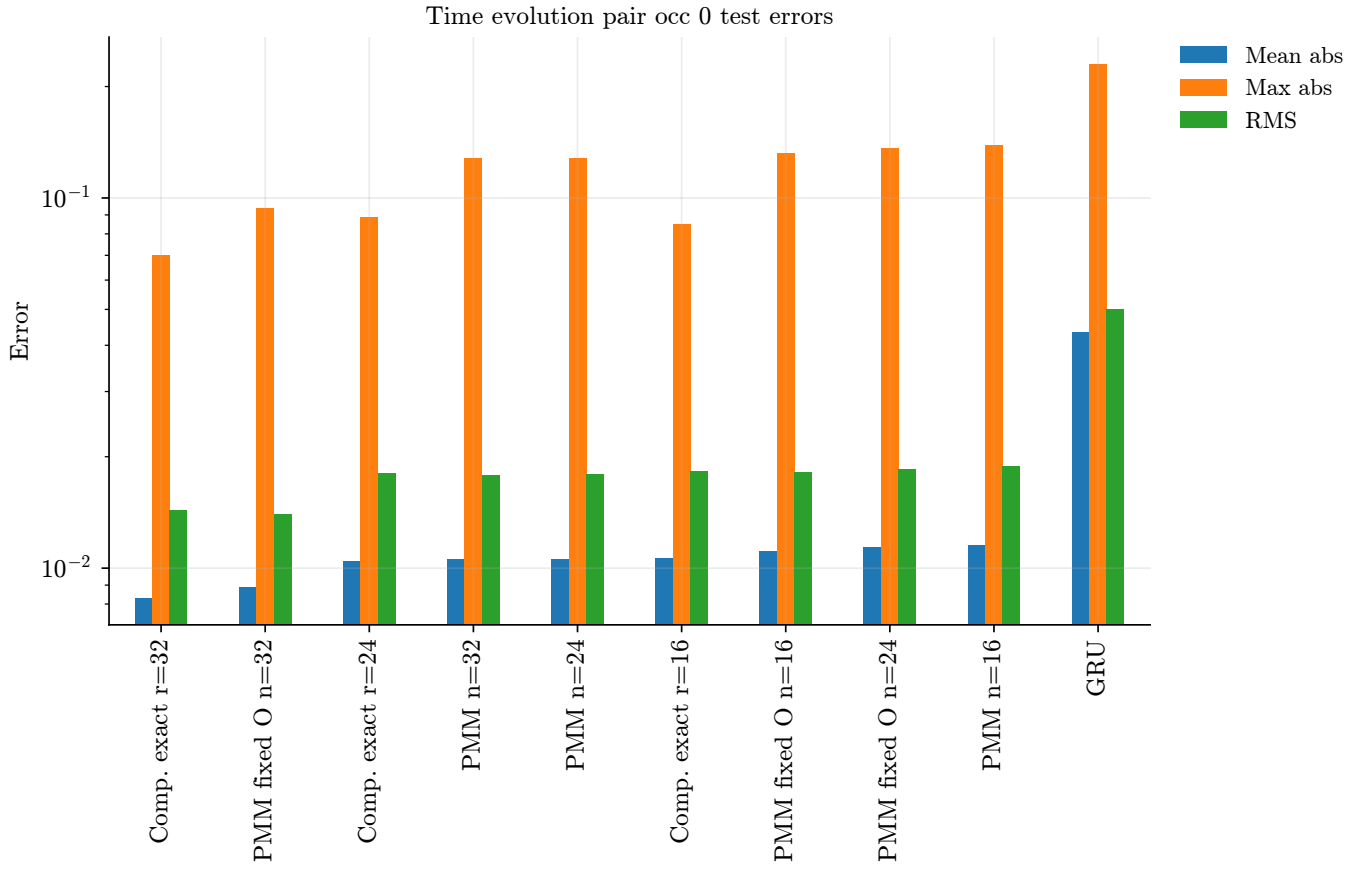

Figure 7. Absolute errors for the time-dependent pair occupation observable in the pairing model. The target observable is the pair occupation at the lowest pair level. The plot compares compressed benchmarks with ranks $r = 16, 24, 32$, PMMs with learned observable readouts and dimensions $n = 16, 24, 32$, PMMs with fixed compressed observables and dimensions $n = 16, 24, 32$, and the best GRU baseline. The learned observable PMMs are labeled as "PMM", while the fixed observable PMMs are labaled as "PMM fixed O". Errors are computed against the full exact observable expectation values on the 48×80 test grid in (g, t).

Model	Dim.	Mean AE	Max AE	RMSE	Params	Train time
Compressed	16	1.06×10^{-2}	8.50×10^{-2}	1.83×10^{-2}	–	–
Compressed	24	1.05×10^{-2}	8.87×10^{-2}	1.81×10^{-2}	–	–
Compressed	32	8.30×10^{-3}	7.00×10^{-2}	1.44×10^{-2}	–	–
PMM fixed O	16	1.11×10^{-2}	1.33×10^{-1}	1.82×10^{-2}	512	66.82
PMM fixed O	24	1.14×10^{-2}	1.36×10^{-1}	1.85×10^{-2}	1152	152.92
PMM fixed O	32	8.89×10^{-3}	9.41×10^{-2}	1.40×10^{-2}	2048	249.59
PMM	16	1.15×10^{-2}	1.39×10^{-1}	1.89×10^{-2}	768	67.81
PMM	24	1.06×10^{-2}	1.28×10^{-1}	1.79×10^{-2}	1728	146.33
PMM	32	1.06×10^{-2}	1.28×10^{-1}	1.78×10^{-2}	3072	230.40
GRU	–	4.34×10^{-2}	2.30×10^{-1}	5.00×10^{-2}	50817	98.27

Table VI. Observable expectation value error metrics for the pairing time-evolution task. In this table, model selection is not included in train time, Again, the learned observable PMM is denoted by "PMM", while the fixed observable PMM is denoted by "PMM fixed O" All errors are computed against the full exact observable expectation values on the 48×80 test grid in (g, t).

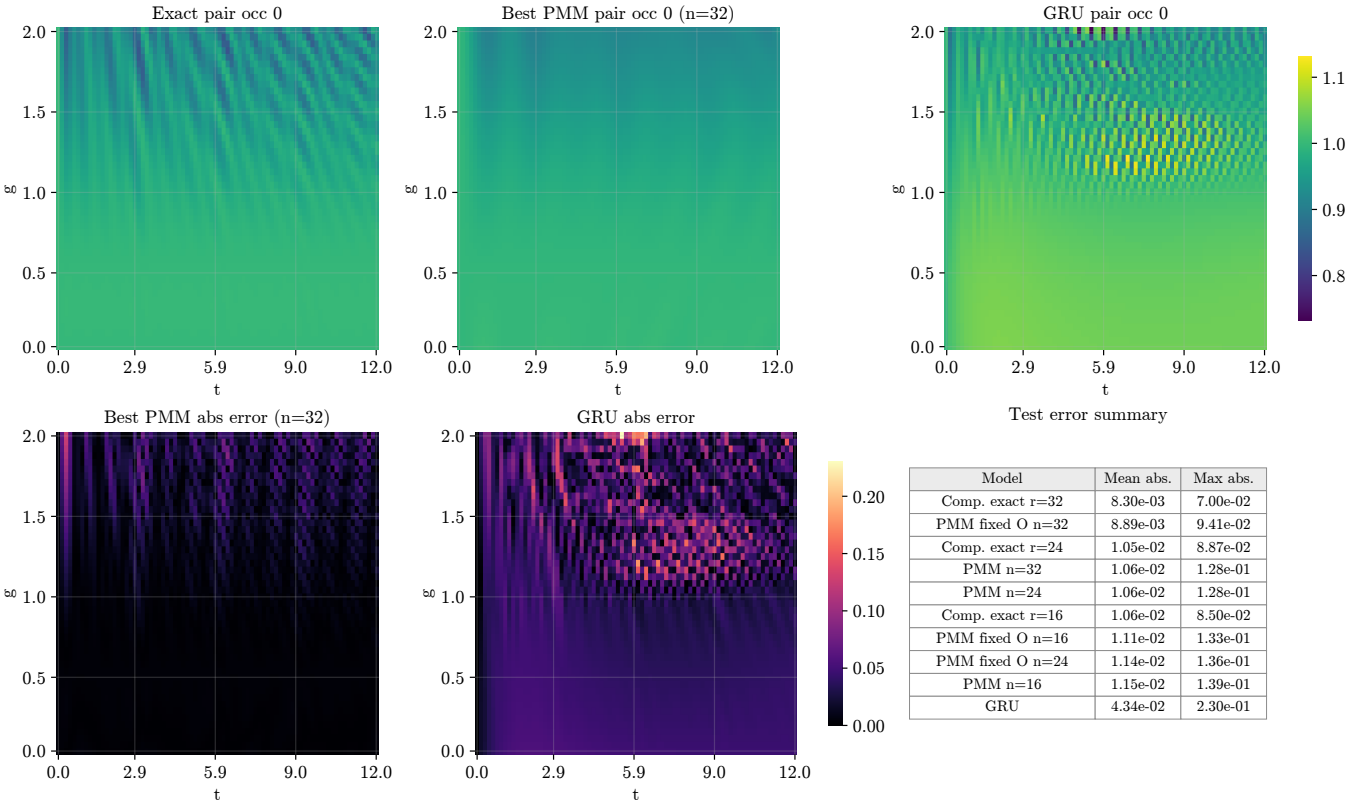

Figure 8. Error heatmaps for the time-dependent pair-occupation observable over the full (g, t) test grid. The figure compares compressed benchmarks with ranks $r = 16, 24, 32$, fixed observable PMMs with dimensions $n = 16, 24, 32$, and the GRU baseline. The color scale shows absolute error relative to the full exact observable expectation values. We include a small table repeating the mean absolute error and the max absolute error, for interpretability of the heatmaps.

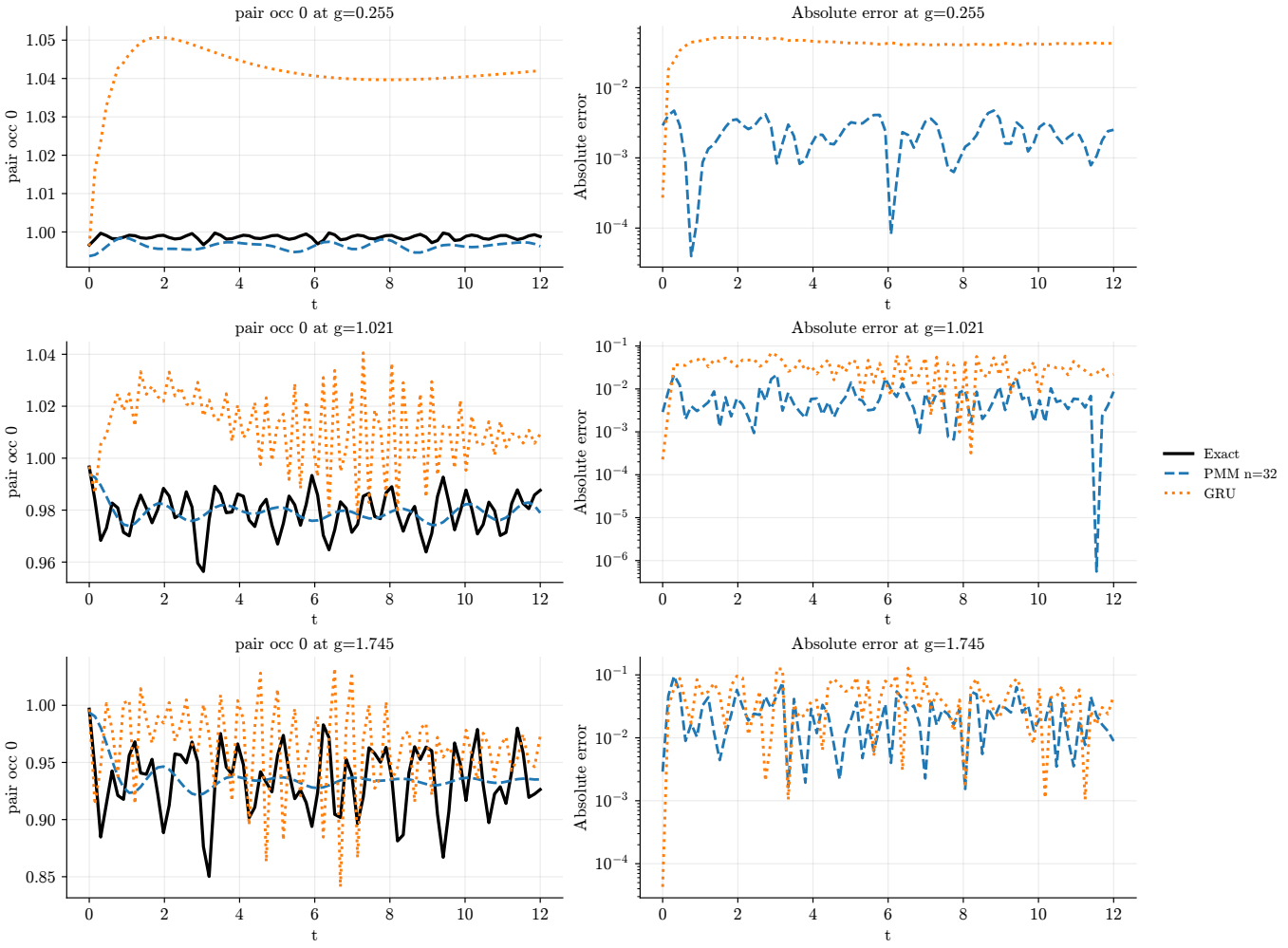

Figure 9. Selected time pointsfor the fixed observable approximation. The plot shows the full exact observable expectation value, the best fixed-observable PMM prediction with $n = 32$, and the GRU baseline at selected values of the interaction strength g.

As can be seen in figure 7, both PMM variants perform substantially better than the GRU baseline. We see that the the fixed and learned observables are quite similar in terms of error, which is a strong sign that the PMM is able to pick up on some of the underlying time-evolution in play. The numerical values in table VI confirms what we saw in figure 7. The best fixed observable PMM, with $n = 32$, is very close to the corresponding compressed benchmark with $r = 32$. The heatmaps in figure 8 show the error structure over the full parameter domain. The PMM errors are generally small, while the GRU error is larger across the whole test grid. As in the survival probability experiment, this is useful because the test grid is denser than the training grid. The result therefore indicates that the PMM generalizes better than the GRU. Finally, figure 9 gives a more direct view of the learned time-evolution by showing selected time points. The PMM seems to learn the general trend of the exact observable time-evolution quite well, while the GRU seems to be able to learn some of the oscillation pattern, but consistently miss the exact figures. This reinforces the previous conclusion, namely that the PMM is better at extracting the important behavior of the time-evolution, while the GRU struggles with generalization.

7. CONCLUSION AND FURTHER WORK

The results show that the performance of the PMMs depends strongly on the model and task. For the Lipkin model, the PMM was able to outperform the neural network baseline in test accuracy, but required relatively large dimensions and much longer training times. This suggests that the Lipkin model is a more demanding test case for the PMM in the parameter regimes considered here. The pairing model gave substantially stronger results. For the static spectral task, the PMMs achieved lower test errors than the neural network baseline while using fewer trainable parameters. Applying compression to the trained PMM surrogate further reduced the effective dimension with only a small additional error, and gave speedups for repeated spectral scans, especially

for the larger PMMs.

The strongest results were obtained for the time-evolution tasks in the pairing model. For the survival probability, the PMMs substantially outperformed the GRU baseline, and the best PMM was close to the corresponding compressed benchmark at matched dimension. The fixed observable experiment gave an even stricter test, and the fact that the fixed observable PMM remained close to the compressed benchmark and clearly outperformed the GRU baseline is one of the main successes of the report.

There are several natural directions for further work. First, the experiments should be repeated with more random seeds and larger training budgets, especially for the time-evolution PMMs, in order to separate optimization effects from approximation effects. Secondly, the compression framework could be studied more systematically, both theoretically and numerically, to understand when low-energy subspaces admit accurate low dimensional representations over parameter domains. Finally, the time-evolution results suggest that PMMs may be useful as low dimensional effective Hamiltonian models for more general time-evolution tasks. A natural next step would be to test the compression in more realistic many body settings. For example, recent work by Jin et al. [13] has used PMMs as surrogate models for QRPA linear response calculations in finite nuclei, where the goal is to approximate expensive observables efficiently. In the original article, the authors use a reduced QRPA emulator to approximate the strength function through a learned low dimensional matrix model, while the compression method could potentially replace this step by constructing the reduced space directly from averaged low energy projectors, possibly giving a more interpretable reduced representation, due to separating the error stemming from dimension reduction and the error stemming from the PMM approximation. Whether the compression methods carries over to a more challenging setting like this is still left to be answered.

[1] D. Dong and I.R. Petersen. Quantum control theory and applications: a survey. *IET Control Theory amp; Applications*, 4(12):2651–2671, December 2010. ISSN 1751-8652. doi:10.1049/iet-cta.2009.0508. URL http://dx.doi.org/10.1049/iet-cta.2009.0508.

[2] Hristiana Atanasova, Liam Bernheimer, and Guy Cohen. Stochastic representation of many-body quantum states. *Nature Communications*, 14(1):3601, 2023. doi:10.1038/s41467-023-39244-4. URL https://doi.org/10.1038/s41467-023-39244-4.

[3] Patrick Cook, Danny Jammooa, Morten Hjorth-Jensen, Daniel D. Lee, and Dean Lee. Parametric matrix models. *Nature Communications*, 16:5929, 2025. doi:10.1038/s41467-025-61362-4. URL https://doi.org/10.1038/s41467-025-61362-4.

[4] Morten Hjorth-Jensen. Second midterm project 2025. FYS4480 course material, University of Oslo, GitHub repository, 2025. URL https://github.com/ManyBodyPhysics/FYS4480/blob/master/doc/Exercises/2025/SecondMidterm2025.pdf. Accessed: 2026-05-27.

[5] Morten Hjorth-Jensen. First midterm project 2025. FYS4480 course material, University of Oslo, GitHub repository, 2025. URL https://github.com/ManyBodyPhysics/FYS4480/blob/master/doc/Exercises/2025/FirstMidterm2025.pdf. Accessed: 2026-05-27.

[6] Michael J. Cervia, A. B. Balantekin, S. N. Coppersmith, Calvin W. Johnson, Peter J. Love, C. Poole, K. Robbins, and M. Saffman. Lipkin model on a quantum computer. *Physical Review C*, 104(2), August 2021. ISSN 2469-9993. doi:10.1103/physrevc.104.024305. URL http://dx.doi.org/10.1103/PhysRevC.104.024305.

[7] Miroslav Hopjan and Lev Vidmar. Scale-invariant survival probability at eigenstate transitions. *Physical Review Letters*, 131(6), August 2023. ISSN 1079-7114. doi:10.1103/physrevlett.131.060404. URL http://dx.doi.org/10.1103/PhysRevLett.131.060404.

[8] Amund Solum Alsvik. Dimensional compression in many body quantum mechanics, 2026. Course project report for FYS5419.

[9] Rajendra Bhatia. *Matrix Analysis*, volume 169 of *Graduate Texts in Mathematics*. Springer, New York, 1997. ISBN 978-1-4612-6857-4. doi:10.1007/978-1-4612-0653-8. Corollary III.1.2.

[10] Amund Solum Alsvik. Fys5429 project code. https://github.com/AmundAlsvik/FYS5429, 2026. GitHub repository, accessed 2026-06-04.

[11] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014. URL https://arxiv.org/abs/1406.1078.

[12] Sebastian Raschka and Vahid Mirjalili. *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2*. Packt Publishing, 3 edition, 2019. ISBN 978-1-78995-575-0.

[13] L. Jin, A. Ravlić, P. Giuliani, K. Godbey, and W. Nazarewicz. Surrogate models for linear response. *Physical Review Research*, 2025. doi:10.1103/vvxs-3mnk. URL https://doi.org/10.1103/vvxs-3mnk.

Appendix A: LLM declaration

1. Tools used

- **ChatGPT** (used during the whole project period).

- **Codex** (used from May–June 2026).

2. Text contributions

Section	Level	Notes
Abstract	1	Grammar and minor sentence structure.
1. Introduction	1	Grammar and minor sentence structure.
2. Physical models and learning tasks	2	Grammar and minor sentence structure. ChatGPT was also used for advice on which learning tasks were good candidates for learning, which ended up becoming the survival probability and the pair occupation observable.
4. The Compression Method	1	Grammar and minor sentence structure.
5. Implementation	2	Grammar and minor sentence structure. ChatGPT was also used for organizing, as well as general structure in the section. It helped me define a sensible break even metric, and it also helped me with formatting the pseudo-code blocks.
6. Results and discussion	2	Grammar and minor sentence structure. ChatGPT also helped with organizing my results and gave advice on which plots and tables to include, and which results to leave in the GitHub repository.
7. Conclusion and further work	1	Grammar and minor sentence structure.

3. Code contributions

File	Level	Description
`common.py`	1	Little LLM use, but Codex/ChatGPT came with suggestions on how to organize the file.
`models.py`	1	Mostly based on code from Morten. Codex/ChatGPT helped with organizing and small corrections
`data.py`	1	Little LLM use, but Codex/ChatGPT came with suggestions on how to organize the file.
`pmm.py`	1	Mostly based on code from Morten. Codex/ChatGPT helped with organizing and small corrections
`compression.py`	2	Codex/ChatGPT was used for generating the function responsible caching system.
`benchmarks.py`	3	Codex/ChatGPT generated the break-even function and helped debugging the other time metric functions.
`baselines.py`	3	Codex/ChatGPT was used to generate a code skeleton based on my original code for the neural network and the GRU.
`time_evolution.py`	3	Codex/ChatGPT generated many of the time evolution functions, in particular for the pair occupation observable.
`lipkin_results.ipynb`	2	Mostly based on code from Morten, but Codex/ChatGPT was used to systemize and help me with expanding the experiments. As well as with organizing the code to avoid duplicate logic.
`pairing_results.ipynb`	2	Essentially the same as the previous. Mostly based on code from Morten, but Codex/ChatGPT/ChatGPT helped with organizing and adapting the code to my use. Codex/ChatGPT also helped with effectivising parts of my code, so that it reused already computed results instead of recomputing.
`time_evolution_results.ipynb`	3	Based on the code skeleton in `time_evolution.py`, Codex/ChatGPT wrote the skeleton for most of the experiments.
`plots.ipynb`	4	Codex/ChatGPT wrote the majority of the file for the plots. I made customization changes and I also verified all the results.

All my experiments were based on a big notebook that had all of my code in it. I used Codex/ChatGPT to sort this notebook into a proper file tree structure, as well as for cleaning up redundancies and dupliate functions. In this process little was changed in terms of computational logic from my original code, so the scores in this table is mostly based on the original code, before Codex systemized it. Codex was also asked to improve docstrings and make them more descriptive.